Institute of Business Administration

NLP with Deep Learning

Assignment 04

Instruction Tuning and Preference Fine-Tuning of Qwen3.5-0.8B-Base

Track 1 — Option A: SFT → DPO Pipeline

Group Members:

Muddasir Javed – 24463
Qamar Raza – 27140
Ramail Khan – 26924

https://github.com/bsparx/NLPPHase2

Submitted To:

Sajjad Haider

May 2026

Contents

1 Introduction **3**

2 Platform Details **3**

3 Data Details **3**
3.1 Evaluation Test Set . . . 4
3.2 SFT Dataset: yahma/alpaca-cleaned . . . 5
3.3 DPO Dataset: trl-lib/ultrafeedback_binarized . . . 5

4 Model and Tokenizer **6**
4.1 Base Model: Qwen/Qwen3.5-0.8B-Base . . . 6
4.2 Tokenizer . . . 6

5 Evaluation Metrics **7**

6 Supervised Fine-Tuning (SFT) **7**
6.1 Training Setup . . . 7
6.2 SFT Trial Configurations . . . 7
6.3 SFT Results . . . 8
6.4 SFT Training Times . . . 8
6.5 Best SFT Trial Selection . . . 9

7 Direct Preference Optimization (DPO) **9**
7.1 Setup . . . 9
7.2 DPO Trial Configurations . . . 10
7.3 DPO Results . . . 10
7.4 DPO Training Times . . . 11
7.5 Best DPO Trial Selection . . . 11

8 Experimentation Analysis and Insights **12**
8.1 Progressive Improvement Across Stages . . . 12
8.2 Impact of LoRA Rank . . . 12
8.3 Impact of Target Modules . . . 13
8.4 Impact of Learning Rate and Epochs . . . 13
8.5 DPO Beta Analysis . . . 13
8.6 Behaviour Differences Across Stages . . . 13
8.7 DPO Reward Signal Analysis . . . 14

9 Output Quality Examples **14**
9.1 Python Coding Task (Prompt 1) . . . 14
9.2 Mathematics Task (Prompt 9) . . . 15
9.3 Failure Cases . . . 16
9.3.1 Response Truncation Due to Model Verbosity . . . 16

9.3.2 Hallucination and Repetition 17

10 Strengths and Weaknesses **18**
10.1 Supervised Fine-Tuning (SFT) 18
10.2 Direct Preference Optimization (DPO) 18

11 Best Model Summary **18**

12 Reproducibility **19**

13 Conclusion **20**

References **21**

1. Introduction

This report documents the complete supervised fine-tuning (SFT) and Direct Preference Optimization (DPO) pipeline applied to `Qwen/Qwen3.5-0.8B-Base`, a compact 0.8-billion-parameter base language model released by Alibaba's Qwen team. The goal is to transform a raw base model into an instruction-following model that produces responses preferred by humans, using parameter-efficient fine-tuning via Low-Rank Adaptation (LoRA).

The pipeline follows **Track 1, Option A**: five SFT trials are first run on an instruction dataset to teach the model the instruction-following format, the best SFT adapter is selected and merged, and five DPO trials are then run on a preference dataset to further align the model towards high-quality responses.

2. Platform Details

All training was conducted on **Modal.com**, a serverless GPU cloud platform that provisions containers on demand. Table 1 summarizes the hardware and software environment.

Table 1: Experimental Platform Configuration

Component	**Details**
Cloud Platform	Modal.com (serverless GPU)
GPU	NVIDIA B200 (Blackwell architecture)
Container OS	Debian Slim (Python 3.11)
PyTorch	$\geq$ 2.2.0
Transformers	$\geq$ 4.51.0 (HuggingFace)
TRL	$\geq$ 0.15.0
PEFT	$\geq$ 0.15.0
Datasets	$\geq$ 3.0.0
Precision	bfloat16 (bf16)
Attention	SDPA (Scaled Dot-Product Attention)
Persistent Storage	Modal Volume (`qwen-adapters`)
Local Machine	Ubuntu 24 (pipeline orchestration only)

The Modal volume `qwen-adapters` persists adapter checkpoints across runs, preventing redundant downloads. The local machine is used only to launch the pipeline and store result JSON files; all GPU computation happens on Modal's remote containers. The SFT and DPO stages each run as separate Modal functions with a 3-hour timeout.

3. Data Details

3.1. Evaluation Test Set

The test set was generated programmatically using **DeepSeek-v4-flash** via the OpenAI-compatible API. A system prompt instructed DeepSeek to produce 10 diverse instruction-following prompts spanning six categories: *coding, science, math, history, creative writing*, and *general knowledge*. DeepSeek then generated gold-standard reference answers for each prompt, serving as ground truth for BLEU and BERTScore evaluation throughout the pipeline.

The use of DeepSeek (rather than the same model used for any judging) satisfies the assignment requirement that gold answers originate from a capable external LLM. Table 2 shows all ten prompts and their categories.

Table 2: Evaluation Test Set — 10 Prompts

ID	Category	Prompt (truncated)
1	Coding	Write a Python function that takes a list of integers and returns the sum of all even numbers...
2	Science	Explain the water cycle, including evaporation, condensation, precipitation, and collection...
3	Math	Solve for x in the equation $2x^2 + 5x - 3 = 0$. Show each step...
4	History	Describe the key events of the American Civil Rights Movement in the 1960s...
5	Creative Writing	Write a 150-word story about a time traveler who accidentally prevents a major historical invention...
6	General Knowledge	List the capital cities of all countries in South America with population of each capital...
7	Coding	Write a JavaScript function that fetches data from a public API and logs the title of the first post...
8	Science	Describe the structure and function of DNA, including the double helix, base pairs, and replication...
9	Math	Calculate the mean, median, and mode of [3, 7, 2, 9, 4, 7, 5, 2, 7]. Show your work...
10	History	Explain the causes and consequences of World War I, focusing on the assassination of Archduke Franz Ferdinand...

3.2. SFT Dataset: yahma/alpaca-cleaned

Table 3: SFT Dataset Details

Property	Value
Dataset name	`yahma/alpaca-cleaned`
Source	HuggingFace Hub
Full dataset size	51,760 instruction-response pairs
Subset used	5,000 samples (random shuffle, seed=42)
Train split	4,500 samples (90%)
Validation split	500 samples (10%)
Format	Instruction / Input (optional) / Output

Alpaca-cleaned is a human-curated and de-duplicated version of the original Stanford Alpaca dataset, which was generated by GPT-3.5 following the self-instruct methodology. It covers general instruction following across diverse topics. We use 5,000 samples rather than the full 51,760 to reduce training time on the B200 GPU while still providing sufficient signal for LoRA fine-tuning of a 0.8B parameter model. Each example is formatted using the ChatML template:

```
text = (
    f"<|im_start|>user\n{instruction}\n\n{input_text}<|im_end|>\n"
    f"<|im_start|>assistant\n{output}<|im_end|>"
)
```

Listing 1: Alpaca ChatML formatting

3.3. DPO Dataset: trl-lib/ultrafeedback_binarized

Table 4: DPO Dataset Details

Property	Value
Dataset name	`trl-lib/ultrafeedback_binarized`
Source	HuggingFace Hub
Full dataset size	62,135 training + 1,000 test
Subset used	2,000 samples (random shuffle, seed=42)
Train split	1,800 samples (90%)
Validation split	200 samples (10%)
Format	Chosen / Rejected conversation pairs

UltraFeedback Binarized is a preference dataset where each entry contains a user prompt along with a chosen (higher-quality) and a rejected (lower-quality) assistant response,

annotated by GPT-4. The conversational structure (list of dialogue turns) requires special parsing to extract the prompt from the first turn and the responses from the final assistant turn. We append `<|im_end|>` to all responses so the model learns when to stop generating.

The 2,000 sample subset was chosen to balance DPO training signal with compute budget (each DPO training step is roughly 2× more expensive than SFT due to the implicit reference forward pass).

4. Model and Tokenizer

4.1. Base Model: Qwen/Qwen3.5-0.8B-Base

Qwen3.5-0.8B-Base is the smallest model in the Qwen3.5 series, with approximately 753 million parameters. It is a decoder-only transformer trained on a large multilingual corpus with a context length of up to 32,768 tokens. As a *base* model (not instruct-tuned), it has no built-in instruction-following capability — responses to prompts are typically continuations of the input text rather than structured answers. This makes it an ideal starting point for studying the effect of SFT and DPO from scratch.

Key architectural properties relevant to LoRA targeting are listed in Table 5.

Table 5: Qwen3.5-0.8B Architecture Highlights

Property	**Value**
Total parameters	≈753 million
Architecture	Decoder-only Transformer
Attention	Multi-head with GQA
Hidden dimension	1,536
Number of layers	28
Attention heads	12
Projection layers	`q_proj, k_proj, v_proj, o_proj`
Vocabulary size	151,936 tokens
Chat template	ChatML (`<|im_start|>` / `<|im_end|>`)

4.2. Tokenizer

The tokenizer is loaded from `Qwen/Qwen3.5-0.8B-Base` using HuggingFace's `AutoTokenizer`. Since the base tokenizer does not define a pad token, we set `pad_token = eos_token` (token ID 151643) to allow batched training. The ChatML special tokens `<|im_start|>` and `<|im_end|>` are native to the vocabulary. During inference, both `eos_token_id` and the `<|im_end|>` token ID are passed as stopping criteria to prevent run-on generation.

5. Evaluation Metrics

Two complementary metrics are used throughout all experiments:

BLEU (Bilingual Evaluation Understudy) measures n-gram precision overlap between the model's generated text and the reference gold answer. Sentence-level BLEU is computed using `sacrebleu.sentence_bleu`, averaged across all 10 test prompts. BLEU captures surface-level lexical similarity and is well-suited for structured outputs like code and mathematical derivations. However, it penalises paraphrasing even when meaning is preserved, making it a conservative lower bound for open-ended tasks.

BERTScore (F1) uses contextualised embeddings from `distilbert-base-uncased` to compute token-level similarity between candidate and reference. It is more robust to synonym substitution and paraphrasing. For this diverse test set spanning creative writing, history, and code, BERTScore is the more reliable primary metric.

Both metrics are computed on the same 10-prompt test set at every stage: base model, after each SFT trial, and after each DPO trial.

> **Limitation note:** With only 10 test prompts spanning highly heterogeneous task types (Python code, algebra, history essays, JavaScript), BLEU scores carry high variance. A coding response may achieve very low BLEU relative to a verbose prose reference even if it is semantically correct. BERTScore is therefore weighted more heavily in model selection decisions.

6. Supervised Fine-Tuning (SFT)

6.1. Training Setup

SFT is performed using HuggingFace's `SFTTrainer` from the TRL library with PEFT's LoRA. The base model is loaded in bfloat16 with SDPA attention. A `DataCollatorForSeq2Seq` is used for dynamic batch padding (rather than always padding to `MAX_SEQ_LENGTH=512`), which significantly reduces wasted computation on short Alpaca examples. Labels corresponding to pad tokens are masked with -100 to exclude them from the cross-entropy loss.

Physical batch size is capped at 4 to prevent CUDA OOM on the B200, with gradient accumulation used to achieve larger effective batch sizes where specified.

6.2. SFT Trial Configurations

Five trials explore a wide range of LoRA ranks, target module sets, learning rates, batch sizes, and epoch counts. Table 6 summarises the configurations.

Table 6: SFT Trial Hyperparameter Configurations

Trial	LoRA r	α	Target Modules	LR	BS	Epochs	Dropout
1	8	16	`q, v`	2e-4	4	1	0.05
2	16	32	`q, v`	2e-4	4	1	0.05
3	8	16	`q, k, v, o`	1e-4	8	1	0.05
4	32	64	`q, v`	5e-5	2	3	0.05
5	16	32	`q, k, v, o`	1e-4	4	2	0.05

All trials use `lora_alpha = 2 × lora_r`, `bias=''none''`, `task_type=CAUSAL_LM`, and bf16 training with the AdamW optimiser and cosine learning rate schedule.

6.3. SFT Results

Table 7 presents BLEU, BERTScore, validation loss, and training time for each SFT trial, along with the base model baseline.

Table 7: SFT Trial Results (Base model baseline: BLEU = 0.1226, BERTScore = 0.8025)

Trial	LoRA r	LR	Epochs	BLEU	BERTScore	Val Loss
Base	—	—	—	0.1226	0.8025	—
1	8	2e-4	1	0.1180	0.8146	1.3594
2	16	2e-4	1	0.1230	0.8127	1.3558
3	8	1e-4	1	0.1182	0.8100	1.3582
4	32	5e-5	3	**0.1256**	0.8144	1.3790
5	16	1e-4	2	0.1138	**0.8169**	**1.3486**

6.4. SFT Training Times

Table 8: Approximate SFT Training Times on B200

Trial	Steps	Approx. Time	Steps/sec
1	563	≈ 9 min	∼1.04
2	563	≈ 9 min	∼1.04
3	563	≈ 9 min	∼1.05
4	1689	≈ 50 min	∼0.57
5	1126	≈ 18 min	∼1.05
Total SFT		**≈ 95 min**	

6.5. Best SFT Trial Selection

Models are ranked by combined score = BLEU + BERTScore, with the lower validation loss used as a tiebreaker for differences smaller than 0.003.

Table 9: SFT Combined Score Ranking

Trial	BLEU + BERTScore	Val Loss	Rank
4	0.9400	1.3790	1
2	0.9357	1.3558	2
5	0.9307	1.3486	3
1	0.9326	1.3594	4
3	0.9282	1.3582	5

Selected: Trial 4 (LoRA r=32, LR=5e-5, BS=2, 3 epochs) with combined score 0.9400. The margin over Trial 2 (0.9357) is $\Delta = 0.0043 \geq 0.003$, so no tiebreaker is triggered and Trial 4 is the clear Rank 1 model. (Note that the tiebreaker is, however, triggered for Trial 1 and Trial 5, where the margin is $0.0019 < 0.003$, successfully ranking Trial 5 higher due to its lower validation loss of 1.3486 vs 1.3594).

Reasoning: The higher rank $r = 32$ provides roughly $4\times$ more trainable parameters than $r = 8$, giving the model more capacity to adapt its attention patterns. Training for 3 epochs rather than 1 allows the model to learn the target format more comprehensively. While this results in a slightly higher validation loss (1.3790 vs ∼1.359 for 1-epoch trials) indicating mild overfitting, it yields the highest combined score on the semantic metrics. The lower learning rate (5e-5 vs 2e-4) avoids overshooting the loss minimum over the additional epochs.

7. Direct Preference Optimization (DPO)

7.1. Setup

The best SFT adapter (Trial 4) is loaded and permanently merged into the base model weights via `PeftModel.merge_and_unload()`. This creates a mathematically clean SFT-merged base from which each DPO trial starts independently. A new LoRA adapter ($r = 16$, $\alpha = 32$, `q_proj` + `v_proj`) is then applied on top and trained with `DPOTrainer`.

DPO with `ref_model=None` uses TRL's implicit reference: the frozen version of the model (LoRA adapters disabled) acts as the reference. Since the SFT weights are baked into the base, the reference effectively represents the SFT model, which is the correct and intended starting point for preference learning.

7.2. DPO Trial Configurations

Five trials vary beta, learning rate, batch size, and epoch count. Table 10 shows all configurations.

Table 10: DPO Trial Hyperparameter Configurations (all trials: LoRA r=16, α=32)

Trial	Beta (β)	LR	BS	Epochs	Seed
1	0.1	5e-6	4	1	42
2	0.5	5e-6	4	1	43
3	0.1	1e-5	4	2	44
4	0.5	1e-5	2	1	45
5	0.2	5e-6	8	1	46

7.3. DPO Results

Table 11 presents the evaluation results for all five DPO trials alongside the SFT and base model baselines.

Table 11: DPO Trial Results

Model	Beta	LR	BLEU	BERTScore	Val Loss
Base	—	—	0.1226	0.8025	—
Best SFT (T4)	—	—	0.1256	0.8144	1.3790
DPO Trial 1	0.1	5e-6	0.1211	0.8142	0.6890
DPO Trial 2	0.5	5e-6	0.1255	0.8175	0.6751
DPO Trial 3	0.1	1e-5	**0.1337**	**0.8230**	**0.6601**
DPO Trial 4	0.5	1e-5	0.1163	0.8133	0.6732
DPO Trial 5	0.2	5e-6	0.1201	0.8112	0.6860

7.4. DPO Training Times

Table 12: Approximate DPO Training Times on B200

Trial	Steps	Approx. Time	Steps/sec
1	225	≈ 5 min	∼0.72
2	225	≈ 5 min	∼0.74
3	450	≈ 10 min	∼0.75
4	225	≈ 9 min	∼0.41
5	225	≈ 5 min	∼0.74
Total DPO		**≈ 34 min**	
Total Pipeline		**≈ 129 min**	

Note: DPO Trial 4 (BS=2) runs at roughly half the throughput of the BS=4 trials due to the reduced parallelism, inflating wall-clock time despite having the same step count.

7.5. Best DPO Trial Selection

Table 13: DPO Combined Score Ranking

Trial	BLEU + BERTScore	Val Loss	Rank
3	**0.9567**	**0.6601**	1
2	0.9430	0.6751	2
5	0.9313	0.6860	3
1	0.9353	0.6890	4
4	0.9296	0.6732	5

Selected: DPO Trial 3 (β=0.1, LR=1e-5, BS=4, 2 epochs) with combined score 0.9567. The margin over Trial 2 (0.9430) is $\Delta = 0.0137 > 0.005$, so no tiebreaker is needed — Trial 3 is the clear winner on both metrics simultaneously.

Reasoning:

- **Low $\beta = 0.1$:** Beta controls the KL penalty against the reference model. A low beta allows the model more freedom to deviate from the SFT distribution and learn stronger preferences. This worked well in conjunction with the higher learning rate.
- **Higher LR = 1e-5:** Compared to the 5e-6 trials (1, 2, 5), the higher learning rate produced larger gradient updates, enabling more meaningful preference learning within the training budget.

- **2 Epochs:** Seeing the preference data twice compounded the learning signal, as evidenced by the steady decline in training loss from epoch 1 (0.690) to epoch 2 (0.665).
- **Tiebreaker Application:** The margin between Trial 1 (0.9353) and Trial 5 (0.9313) is $\Delta = 0.0040 < 0.005$. Applying the tiebreaker rule, Trial 5 is correctly ranked 3rd due to its lower validation loss (0.6860 vs 0.6890).
- **Trial 4 failure:** Same LR but β=0.5 over-constrained the updates — the high KL penalty forced the model to stay close to the reference, limiting how much it could shift toward preferred outputs.

8. Experimentation Analysis and Insights

8.1. Progressive Improvement Across Stages

Table 14 shows the overall progression from base model to final DPO model.

Table 14: Score Progression Across Pipeline Stages

Stage	BLEU	BERTScore	Δ BLEU	Δ BERT
Base Model	0.1226	0.8025	—	—
Best SFT (Trial 4)	0.1256	0.8144	+0.0030	+0.0119
Best DPO (Trial 3)	0.1337	0.8230	+0.0111	+0.0205

The pipeline shows monotonic improvement at each stage. BERTScore improved by +2.05 points absolute over the base model after the full SFT+DPO pipeline, indicating that the model's outputs are semantically much closer to the gold answers. The DPO stage contributes substantially (+0.0081 BLEU, +0.0086 BERTScore over SFT alone), validating that preference optimization adds signal beyond supervised instruction tuning.

8.2. Impact of LoRA Rank

Across SFT trials, increasing LoRA rank generally improved performance:

- $r = 8$ (Trials 1, 3): Combined scores 0.9326, 0.9282
- $r = 16$ (Trials 2, 5): Combined scores 0.9357, 0.9307
- $r = 32$ (Trial 4): Combined score 0.9400

Higher rank gives the LoRA adapters more representational capacity. However, $r = 32$ adds ∼5.50M trainable parameters versus ∼1.38M for $r = 8$, increasing memory usage. The improvement from $r = 8$ to $r = 32$ is modest (∼0.007 combined score), suggesting diminishing returns beyond $r = 16$ for a 0.8B parameter model at this dataset scale.

8.3. Impact of Target Modules

Trials 3 and 5 added `k_proj` and `o_proj` alongside `q_proj` and `v_proj`. Surprisingly, expanding to all four projection matrices did not improve performance over the two-module `q, v` configuration. Trial 3 ($r = 8$, `q,k,v,o`) scored lower than Trial 1 ($r = 8$, `q,v`). This is likely because for a small dataset of 5,000 samples, adding more adapter parameters increases the risk of overfitting without proportional gains in task alignment.

8.4. Impact of Learning Rate and Epochs

Trial 4's combination of a low learning rate (5e-5) with 3 epochs produced the best SFT result. High learning rates (2e-4 in Trials 1 and 2) risk overshooting with LoRA because the adapter parameters have much smaller effective scale. Longer training with a conservative rate consistently beats aggressive single-epoch training on this dataset size.

8.5. DPO Beta Analysis

The DPO beta parameter (β) controls the trade-off between maximising preference rewards and staying close to the reference (SFT) distribution:

- $\beta = 0.1$ (low): Trials 1, 3 — low constraint, allows model to deviate further from SFT
- $\beta = 0.2$ (medium): Trial 5 — intermediate constraint
- $\beta = 0.5$ (high): Trials 2, 4 — strong KL penalty, forces proximity to reference

At $\beta = 0.5$, Trial 4 (LR=1e-5) performed *worse* than Trial 2 (LR=5e-6) despite the higher learning rate — suggesting that strong KL penalties interact poorly with large gradient updates, potentially causing instability in the reward signal. The reward margins in Trial 4's eval logs were among the highest (0.1095), yet the generation quality decreased, possibly because the model is trading off response quality for higher reward-model scores.

8.6. Behaviour Differences Across Stages

Base model: As a raw base model, Qwen3.5-0.8B-Base does not natively follow instructions. Given the ChatML prompt format, it may continue generating incomplete thoughts, repeat the question, or produce text in a non-answer format. BLEU of 0.1226 and BERTScore of 0.8025 reflect partial overlap that happens to exist between its continuations and the reference answers.

After SFT: The model learns to respond in the ChatML format — generating text after the `<|im_start|>assistant` token and terminating at `<|im_end|>`. Responses become more structured and topically relevant. BERTScore improves consistently across all 5 trials (+0.0075 to +0.0144), confirming that the model is now semantically engaging with the question.

After DPO: Response quality is further refined toward human-preferred outputs. The

model produces more complete, well-organised answers. The DPO training on UltraFeedback (which contains GPT-4 preference annotations) exposes the model to a wide variety of instruction types and teaches it to prefer longer, more detailed, and more accurate responses over shorter or evasive ones.

8.7. DPO Reward Signal Analysis

From the DPO training logs, the `eval_rewards/accuracies` metric tracks how often the model assigns higher log-probability to chosen responses than rejected ones:

Table 15: DPO Reward Accuracy and Margins at Eval

Trial	Reward Accuracy	Reward Margin	Final Eval Loss
1	0.505	0.010	0.689
2	0.515	0.074	0.675
3	0.600	0.086	0.660
4	0.540	0.110	0.673
5	0.470	0.022	0.686

Trial 3 achieves the highest reward accuracy (60%) and a substantial reward margin (0.086), confirming genuine preference learning. Trial 5's reward accuracy of 0.47 (below chance) suggests that 1 epoch at the low learning rate 5e-6 with BS=8 was insufficient to establish a reliable preference signal.

9. Output Quality Examples

The following examples show the actual outputs produced by the best SFT model (Trial 4) and best DPO model (Trial 3) on selected test prompts, as recorded during evaluation.

9.1. Python Coding Task (Prompt 1)

Prompt: *Write a Python function that takes a list of integers and returns the sum of all even numbers in the list. Include an example usage.*

Base Model Output

[Base model generates a raw continuation without the ChatML assistant format; response does not complete a structured function and terminates without example usage.]

Best SFT Model Output — Trial 4 (Actual Prediction)

Here is a Python function that takes a list of integers and returns the sum of all even numbers in the list:

```
def sum_even_numbers(numbers):
return sum([num for num in numbers if num % 2 == 0])
```

Example usage:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(sum_even_numbers(numbers))  # Output:  24
```

In this example, the function `sum_even_numbers` takes a list of integers as input and returns the sum of all even numbers in the list. In this case, the input list is `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`, and the output is `24`, which is the sum of all even numbers in the list.

Best DPO Model Output — Trial 3 (Actual Prediction)

Here's a Python function that takes a list of integers and returns the sum of all even numbers in the list:

```
def sum_even_numbers(numbers):
return sum([num for num in numbers if num % 2 == 0])
```

Example usage:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(sum_even_numbers(numbers))  # Output:  24
```

In this example, the function `sum_even_numbers` takes a list of integers as input and returns the sum of all even numbers in the list. In this case, the input list is `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`, and the output is `24` because the sum of the even numbers in the list is `2 + 4 + 6 + 8 = 20`, and the sum of the odd numbers is `1 + 3 + 5 + 7 + 9 = 25`, and the total sum is `20 + 25` ... *[output truncated at 256 tokens]*

Both the SFT and DPO models produce a functionally correct Python function — the list comprehension with a modulo check is valid. After SFT, the model responds in the correct ChatML assistant format with a clean function and example, completing within the token budget. After DPO, the model attempts a more detailed explanation of the arithmetic breakdown, reflecting the preference signal for more thorough responses, but is cut off by the 256-token generation cap before completing. Both models annotate the example with `# Output:  24`, which is incorrect for `[1...10]` (the correct even sum is 30); this arithmetic error in the comment is a residual hallucination unresolved by either training stage.

9.2. Mathematics Task (Prompt 9)

Prompt: *Calculate the mean, median, and mode of [3, 7, 2, 9, 4, 7, 5, 2, 7]. Show your work.*

Both the SFT Trial 4 and DPO Trial 3 models produce nearly identical, well-structured

step-by-step responses. Both correctly sort the dataset, identify the median as the 5th element (5), and identify mode = 7 (appearing 3 times). Both conclude with an explicit labelled summary block reproduced below:

Best SFT and DPO Models — Summary Block (identical across both)

Summary:
- Mean: 5.33
- Median: 5
- Mode: 7

The primary shared error is arithmetic: both models compute `Sum = 48` instead of the correct value of 46, yielding `Mean = 5.33` instead of the correct `5.11`. This miscalculation is consistent across all ten SFT and DPO trials (see Section 9.3). The improvement DPO contributes here is the reliable presence of the structured summary block — which SFT Trial 4 also produces but which was absent in several earlier SFT trials (e.g., Trial 1).

9.3. Failure Cases

9.3.1. Response Truncation Due to Model Verbosity

All evaluations were conducted with `max_new_tokens=256`. Importantly, all ten gold answers are concise and well within this limit (the longest, the creative writing story, is approximately 200 words). The truncation issue therefore does *not* stem from the gold answers being too long. Instead, it arises because the fine-tuned models tend to generate verbose, padded, or repetitive outputs that exhaust the 256-token budget before reaching a natural conclusion.

This is a consistent constraint applied equally to all models — base, SFT, and DPO — so relative rankings between trials remain valid. However, for prompts where the model generates correctly but verbosely, truncation means the BLEU comparison is between a partial prediction and a complete gold answer, depressing absolute scores.

Truncation note: The 256-token generation cap does not affect the gold answers (which are all complete and short). It affects *model outputs* that become verbose or enter repetition loops, causing them to be cut off mid-response. Relative trial rankings are unaffected since all models were evaluated identically. The root cause of most truncations is model verbosity and hallucination loops, not insufficient token budget.

Table 16: Prompts Where Model Output Was Truncated at 256 Tokens

Prompt	Category	Observed Truncation Behaviour
2	Science (water cycle)	Model generates verbose stage-by-stage breakdown and cuts off mid-sentence during Collection
5	Creative writing	Repetition loops consume tokens; story never reaches a resolution
6	General knowledge	Model repeats capital entries in a loop, exhausting budget before listing all 12 countries
8	Science (DNA)	Verbose structural description runs out of tokens during replication explanation

9.3.2. Hallucination and Repetition

- **Capital city list (Prompt 6):** Beyond truncation, all SFT trials hallucinate population figures and repeat the same entries multiple times. Trial 1 lists "Buenos Aires" three times with the same population of 3.5 million; Trial 3 lists "Brasília" eight consecutive times. This is a classic small-model repetition failure where the model learns the list format but loses track of which entries it has already generated.
- **Creative writing (Prompt 5):** All SFT trials replace the historically accurate Gutenberg printing-press context with anachronistic references to Thomas Edison and light bulbs, indicating the model conflates "famous inventor" with its most frequent training example. Trial 2 enters a severe repetition loop: *"The printer was delighted with the results and asked Thomas Edison to try it out on a larger scale"* repeats verbatim six times.
- **JavaScript async (Prompt 7):** SFT Trials 1, 3, and 5 generate a non-async `fetch()` call without `await`, then fill the function body with repeated `console.log(response.hea` lines — a clear hallucination loop. The model learned the `fetch` API surface but not the async/await pattern.
- **Mathematics (Prompt 9):** All SFT and DPO trials compute the dataset sum incorrectly as 48 instead of the correct 46, yielding mean = 5.33 instead of 5.11. The model pattern-matches the arithmetic format (sum / count) but miscounts the addition. The median and mode are computed correctly in all trials. Additionally, SFT Trial 2 further misidentifies the median as 4 rather than the correct value of 5.
- **Long-form history (Prompts 4, 10):** All models paraphrase rather than reproduce specific facts (exact act names, dates, march routes), which systematically lowers BLEU even when responses are factually reasonable. Trial 3's WWI response (Prompt 10) enters a severe hallucination loop, repeating "the assassination of Archduke Franz Ferdinand's wife, Sophie" four times in succession.

10. Strengths and Weaknesses

10.1. Supervised Fine-Tuning (SFT)

Table 17: SFT: Strengths and Weaknesses

Strengths	Weaknesses
Teaches the model to follow the instruction format quickly with minimal data (5k samples)	Does not distinguish between high-quality and low-quality outputs
Stable, predictable training with well-understood cross-entropy loss	Model may memorise dataset patterns rather than generalising
Computationally cheap with LoRA (only 0.04%–0.17% of params trained)	Requires a well-formatted dataset; noisy instructions degrade output quality
Effective first step before DPO — necessary to establish a sensible reference distribution	Gains plateau quickly; additional epochs beyond ∼3 yielded diminishing returns

10.2. Direct Preference Optimization (DPO)

Table 18: DPO: Strengths and Weaknesses

Strengths	Weaknesses
Directly optimises for human preference without a separate reward model	Sensitive to β choice: too high prevents learning, too low risks reward hacking
Consistently improves over SFT on both BLEU and BERTScore in this experiment	Requires a high-quality preference dataset with clearly distinguishable chosen/rejected pairs
Implicit reference model (`ref_model=None`) saves memory and avoids double model loading	Training loss ∼0.69 near log(2) suggests the model is close to random at distinguishing preferences in some trials
Can be layered on top of the SFT-merged model cleanly via LoRA	Less interpretable than SFT: it is not obvious which specific behaviours are being reinforced

11. Best Model Summary

Table 19 summarises the final selected models and their complete configurations.

Table 19: Best Model Configurations

Parameter	**Best SFT (Trial 4)**	**Best DPO (Trial 3)**
Base Model	Qwen3.5-0.8B-Base	Qwen3.5-0.8B-Base + SFT T4
LoRA Rank r	32	16
LoRA Alpha	64	32
Target Modules	q_proj, v_proj	q_proj, v_proj
LoRA Dropout	0.05	0.05
Learning Rate	5e-5	1e-5
Batch Size	2 (eff. 8 w/ accum.)	4
Epochs	3	2
Beta (β)	N/A	0.1
Dataset	alpaca-cleaned (5k)	ultrafeedback_binarized (2k)
BLEU	0.1256	0.1337
BERTScore	0.8144	0.8230
Val Loss	1.3790	0.6601
Adapter Path	`sft_trial_4_adapter`	`dpo_trial_3_adapter`

12. Reproducibility

All experiments are fully reproducible. The entire codebase is hosted on GitHub at https://github.com/bsparx/NLPPHase2. The following steps recreate the entire pipeline from scratch:

1. **Install dependencies:**

```
pip install modal openai
modal setup    # authenticate Modal account
```

2. **Generate the test set:**

```
export DEEPSEEK_API_KEY='<your_key>'
python generate_test_set.py
# Output: test_set.json (10 prompts + gold answers from
   DeepSeek-v4-flash)
```

3. **Run the full SFT → DPO pipeline on Modal:**

```
modal run modal_train.py
# SFT and DPO each run as separate Modal B200 functions.
# Results saved locally as sft_results.json and dpo_results.
   json.
```

4. **Resume from cached SFT results:** If `sft_results.json` already exists locally, the pipeline skips SFT and proceeds directly to DPO, loading the cached best

adapter name.

5. **Download adapters:**

```
modal volume get qwen-adapters / local_adapters/
```

Seeds: All trials use fixed seeds (42–46 for SFT, 42–46 for DPO) passed to `SFTConfig`/`DPOConfig`, ensuring deterministic training within the same container.

Dataset reproducibility: Both datasets are loaded from HuggingFace Hub with fixed `shuffle(seed=42)` and `select(range(N))` calls. The same 5,000 SFT and 2,000 DPO samples are used in every run.

Model reproducibility: The base model is cached to the Modal volume on first download and reused for all subsequent trials, preventing version drift.

13. Conclusion

This assignment successfully implemented a complete SFT → DPO fine-tuning pipeline on `Qwen/Qwen3.5-0.8B-Base` using parameter-efficient LoRA adapters. The pipeline ran on Modal.com's B200 GPU in approximately 130 minutes total.

The key findings are:

- **SFT** effectively teaches instruction-following format to the base model. BERTScore improved across all 5 trials (+0.0075 to +0.0144), while BLEU showed mixed results, reflecting the metric's sensitivity to exact phrasing.
- **DPO** adds meaningful improvement over SFT: the best DPO model achieves BLEU=0.1337 (+9.1% over base) and BERTScore=0.8230 (+2.6% over base).
- **Hyperparameter sensitivity** is significant: the difference between the best and worst DPO trials spans 0.027 BERTScore, underlining the importance of systematic search.
- **Low beta (0.1) + moderate-high LR (1e-5) + 2 epochs** is the optimal DPO configuration for this model and dataset scale.
- The pipeline is fully reproducible with fixed seeds, cached model weights, and deterministic dataset selection.

References

1. Qwen Team. *Qwen 3.5 0.8B model card.* Alibaba Group, 2026. https://huggingface.co/Qwen/Qwen3.5-0.8B-Base

2. Yahma. (2023). *alpaca-cleaned* (dataset). https://huggingface.co/datasets/yahma/alpaca-cleaned

3. Cui, G., et al. (2023). *UltraFeedback: Boosting Language Models with High-quality Feedback.* arXiv preprint arXiv:2310.01377. https://huggingface.co/datasets/trl-lib/ultrafeedback_binarized

4. Modal Labs. *Modal Documentation.* https://modal.com/docs